

HDOC - Day 16

Digit Ordering and Largest/Smallest Operations Using Void Functions

Algorithm

1. Start. 2. Display menu and read choice. 3. Call one separate parameterless void function for each option. 4. Each function reads its own positive integer. 5. Increasing/decreasing functions compare adjacent digits while processing the number. 6. largestSmallest() tracks maximum and minimum digits. 7. secondDigits() records which digits occur and finds the second distinct smallest and largest digits. 8. Repeat until Exit. 9. Stop.

Test Case Table

TC	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	12345; option 1	Increasing order	Increasing order	Pass
2	97531; option 2	Decreasing order	Decreasing order	Pass
3	50826; option 3	Largest=8, Smallest=0	8, 0	Pass
4	57294; option 4	2nd largest=7, 2nd smallest=4	7, 4	Pass

C Program

```
#include <stdio.h>
void increasing(void); void decreasing(void); void largestSmallest(void); void secondDigits(void);
int main() {
    int ch;
    do {
        printf("\n1. Increasing order\n2. Decreasing order\n3. Largest and smallest\n");
        printf("4. Second largest and second smallest\n5. Exit\nEnter choice: ");
        scanf("%d",&ch);
        switch(ch) {
            case 1: increasing(); break; case 2: decreasing(); break;
            case 3: largestSmallest(); break; case 4: secondDigits(); break;
            case 5: break; default: printf("Invalid choice.\n");
        }
    } while(ch!=5); return 0;
}

void increasing(void) {
    int n,prev,d,ok=1; printf("Enter a positive integer: "); scanf("%d",&n);
    prev=10;
    while(n>0) { d=n%10; if(d>=prev) ok=0; prev=d; n/=10; }
    printf(ok?"Digits are in increasing order\n":"Digits are not in increasing order\n");
}

void decreasing(void) {
    int n,prev=-1,d,ok=1; printf("Enter a positive integer: "); scanf("%d",&n);
    while(n>0) { d=n%10; if(d<=prev) ok=0; prev=d; n/=10; }
    printf(ok?"Digits are in decreasing order\n":"Digits are not in decreasing order\n");
}

void largestSmallest(void) {
    int n,d,large=0,small=9; printf("Enter a positive integer: "); scanf("%d",&n);
    while(n>0){ d=n%10; if(d>large)large=d; if(d<small)small=d; n/=10; }
    printf("Largest digit = %d\nSmallest digit = %d\n",large,small);
}

void secondDigits(void) {
    int n,d,i,f[10]={0},s1=-1,s2=-1;
    printf("Enter a positive integer: "); scanf("%d",&n);
    while(n>0){ f[n%10]=1; n/=10; }
    for(i=0;i<=9;i++) if(f[i]) { if(s1==-1)s1=i; else if(s2==-1)s2=i; }
    int l1=-1,l2=-1;
    for(i=9;i>=0;i--) if(f[i]) { if(l1==-1)l1=i; else if(l2==-1){l2=i;break;} }
    if(s2==-1 || l2==-1) printf("Second largest/smallest not available\n");
    else printf("Second largest = %d\nSecond smallest = %d\n",l2,s2);
}
```

Compilation, Execution & Testing

```
puppie@Anvikas-MacBook-Air Desktop % clang day16.c -o day16
puppie@Anvikas-MacBook-Air Desktop % ./day16
Enter choice: 1
Enter a positive integer: 12345
Digits are in increasing order
```

```
puppie@Anvikas-MacBook-Air Desktop % ./day16
Enter choice: 2
Enter a positive integer: 97531
Digits are in decreasing order
```

```
puppie@Anvikas-MacBook-Air Desktop % ./day16
Enter choice: 3
Enter a positive integer: 50826
Largest digit = 8
Smallest digit = 0
```

```
puppie@Anvikas-MacBook-Air Desktop % ./day16
Enter choice: 4
Enter a positive integer: 57294
Second largest = 7
Second smallest = 4
```

Result: The menu-driven C program was compiled, executed and tested successfully against the prepared test cases.